

arc-agi

Taxonomía Deep Learning con Benchmarks ARC-AGI

Deep Learning
├─ 1. Modelos Perceptivos
│ ├── CNNs
│ ├── Vision Transformers (ViT)
│ └── Multimodal Encoders
├─ 2. Modelos Generativos
│ ├── GANs
│ ├── VAEs
│ ├── Diffusion Models
│ └── Flow-based Models
├─ 3. Modelos Secuenciales
│ ├── RNN / LSTM / GRU
│ ├── Transformers
│ └── State Space Models (Mamba, S4)
├─ 4. Foundation Models (Base LLMs)
│ ├── LLMs
│ │ ├── GPT-3 ─────────────────── ARC-AGI-1: ~0%
│ │ ├── GPT-4 ─────────────────── ARC-AGI-1: ~0%
│ │ ├── GPT-4o ─────────────────── ARC-AGI-1: 5%
│ │ ├── GPT-4.1-Nano ─────────── ARC-AGI-1: 0% \$0.002/task
│ │ ├── Llama 4 Maverick ───────── ARC-AGI-1: 4.4% \$0.0078/task ARC-AGI-2: 0%
│ │ ├── Grok 3 ─────────────────── ARC-AGI-1: 5.5% \$0.0093/task ARC-AGI-2: 0%
│ │ └── Claude 3.5 Sonnet ───────── ARC-AGI-1: 14% (base, sin técnicas adicionales)
│ ├── Vision Foundation Models
│ └── Multimodal Foundation Models
├─ 5. Neuro-Symbolic & Reasoning AI
│ ├── Neuro-symbolic models
│ ├── Program Induction Models
│ │ ├── Test-Time Training (MIT/Cornell) ─ ARC-AGI-1: 47.5%
│ │ └── CompressARC (76K params) ───────── ARC-AGI-1: 20-34% ARC-AGI-2: 4% (sin pretraining)
│ (https://iliao2345.github.io/blog_posts/arc_agi_without_pretraining/arc_agi_without_pretraining.html)
│ ├── Neural Theorem Provers
│ └── Differentiable Reasoning Systems
├─ 6. World Models & Cognitive Models
│ ├── World Models
│ ├── Predictive Processing Models
│ ├── Active Inference Models
│ └── Cognitive Architectures
├─ 7. **Reasoning-Centric Models (LRMs - Large Reasoning Models)**
│ ├── TRM (Token Reasoning Models) - Single Chain-of-Thought
│ │ ├── OpenAI o1 (low) ─────────── ARC-AGI-1: 20.5% \$0.43/task
│ │ ├── OpenAI o1 (medium) ───────── ARC-AGI-1: 31% \$0.79/task
│ │ ├── OpenAI o1 (high) ─────────── ARC-AGI-1: 35% \$1.31/task
│ │ ├── DeepSeek R1-Zero ─────────── ARC-AGI-1: 14% \$0.11/task (sin SFT, solo RL)
│ │ ├── DeepSeek R1 ───────────────── ARC-AGI-1: 15.8% \$0.06/task ARC-AGI-2: 1.3%
│ │ ├── DeepSeek R1 (5/28/25) ─────── ARC-AGI-1: 21.2% \$0.046/task (actualizado)
│ │ ├── OpenAI o3 (low) ─────────── ARC-AGI-1: 41% \$1.22/task ARC-AGI-2: 1.9%
│ │ ├── OpenAI o3 (medium) ───────── ARC-AGI-1: 53% \$2.52/task ARC-AGI-2: 2.9%
│ │ ├── OpenAI o3 (high) ─────────── ARC-AGI-1: 60.8% \$0.50/task ARC-AGI-2: 6.5%
│ │ ├── OpenAI o3-pro (high) ─────── ARC-AGI-1: 59.3% \$4.16/task ARC-AGI-2: 4.9%
│ │ ├── OpenAI o4-mini (low) ─────── ARC-AGI-1: 21% \$0.05/task ARC-AGI-2: 1.6%
│ │ ├── OpenAI o4-mini (medium) ───── ARC-AGI-1: 42% \$0.23/task ARC-AGI-2: 2.3%
│ │ ├── OpenAI o4-mini (high) ─────── ARC-AGI-1: 58.7% \$0.406/task ARC-AGI-2: 6.1%
│ │ ├── Grok 3 Mini (low) ─────────── ARC-AGI-1: 16.5% \$0.0099/task ARC-AGI-2: 0.4%
│ │ ├── Claude Sonnet 4 (16k) ─────── ARC-AGI-1: 40% \$0.366/task ARC-AGI-2: 5.9%
│ │ ├── Claude Opus 4 (16k) ───────── ARC-AGI-1: 35.7% \$1.25/task ARC-AGI-2: 8.6%
│ │ ├── Claude Opus 4.5 (64k) ─────── ARC-AGI-2: 37.6% \$2.20/task SOTA comercial verificado
│ │ ├── Gemini 2.5 Flash ─────────── ARC-AGI-1: 33.3% \$0.037/task (mejor balance)
│ │ ├── Gemini 2.5 Pro ─────────────── ARC-AGI-1: 33% \$0.569/task ARC-AGI-2: 3.8%
│ │ └── Gemini 3 Pro ───────────────── ARC-AGI-2: 31% \$0.81/task (baseline)
│ └── Search-Augmented Reasoning Models (Test-Time Compute Scaling)

```
| | | └─ OpenAI o3-preview (low) ─ ARC-AGI-1: 75.7% | $200/task | 33.5M tokens
| | | └─ OpenAI o3-preview (high) ─ ARC-AGI-1: 87.5% | $4,560/task | 5.7B tokens (172x compute)
| | | └─ Claude 3.5 Sonnet + Evolutionary TTC ─ ARC-AGI-1: 53.6% (Jeremy Berman)
|
| └─ **Tiny Recursive Models (TRM) - Redes Pequeñas con Razonamiento Recursivo** NEW
| | └─ TRM (7M params) ───────── ARC-AGI-1: 45% | ARC-AGI-2: 8% | Paper Award 1st 2025
| | (https://arxiv.org/abs/2510.04871)
| | | └─ Solo 2 capas, <0.01% parámetros de LLMs, supera DeepSeek R1, o3-mini, Gemini 2.5 Pro
| | | └─ HRM (27M params) ───────── ARC-AGI-1: ~40% | Hierarchical Reasoning Model (precursor)
|
| └─ HRM (Hierarchical Reasoning Models)
| | └─ Hierarchical planners
| | └─ Modular reasoning networks
| | └─ Multi-level cognitive models
|
└─ 8. Agentic AI Systems
  └─ Autonomous Agents
  └─ Tool-using Models
    └─ SLM Fine-tuned (OPT-350M) ─ ToolBench: 77.55% (supera ChatGPT-CoT 26%)
  └─ Planning Agents
  └─ Multi-agent Systems

└─ 9. **Soluciones Híbridas / Especializadas (Kaggle ARC Prize)**
  └─ ARC Prize 2024 (ARC-AGI-1)
    └─ the ARCHitects (1°) ─── 53.5% (Program synthesis + search) (https://da-fr.github.io/arc-prize-2024/the_architects.pdf) usando x4 NVIDIA T4
    └─ Guillermo Barbadillo (2°) ─ 40%
    └─ alijs (3°) ───────── 40%
    └─ William Wu (4°) ─── 37%
    └─ Ensemble Solutions ─── ~81% (combinación de múltiples soluciones)
  └─ ARC Prize 2025 (ARC-AGI-2) (https://arxiv.org/pdf/2601.10904)
    └─ NVARC (1°) ───────── 24.03% | $0.20/task | SOTA Kaggle
    | (https://drive.google.com/file/d/1vkEluuaaJTzaZiJL69TkZovJUkPSDH5Xc/view)
    └─ the ARCHitects (2°) ─── 16.53% | 2D-aware masked-diffusion LLM
    | (https://lambdalabsml.github.io/ARC2025_Solution_by_the_ARCHitects/)
    └─ MindsAI (3°) ───────── 12.64% | Test-time training pipeline (https://arxiv.org/abs/2506.14276)
    └─ Lonnie (4°) ───────── 6.67%
    | (https://www.kaggle.com/competitions/arc-prize-2025/writeups/arc-prize-2025-competition-writeup-5th-place)
    └─ G. Barbadillo (5°) ─── 6.53%
    | (https://ironbar.github.io/arc25/05_Solution_Summary/#vision-search-and-learn)
  └─ 10. **Model Refinement Solutions (Refinement Loops)**
    └─ Poetiq (Gemini 3 Pro Ref) ─ ARC-AGI-2: 54% | $31/task | SOTA refinement
    └─ Poetiq (Claude Opus 4.5 Ref) ─ ARC-AGI-2: ~54% | $60/task
    └─ SOAR (Self-Improving LLM) ─ ARC-AGI-1: 52% | Paper Award 2nd 2025
    └─ Evolutionary Program Synthesis (E. Pang) ─ Paper Award Runner-up
```

Resumen de Benchmarks ARC-AGI (Actualizado Dic 2025)

ARC-AGI-1 Semi-Private Eval (Ranking por rendimiento)

Modelo/Sistema	Score	Costo/Tarea	Notas
o3-preview (high)	87.5%	\$4,560	172x compute, 5.7B tokens
Ensemble Kaggle	~81%	-	Combinación de soluciones
o3-preview (low)	75.7%	\$200	Primer puesto leaderboard 2024
o3 (high)	60.8%	\$0.50	-
o3-pro (high)	59.3%	\$4.16	-
o4-mini (high)	58.7%	\$0.406	-
Claude 3.5 + EvoTTC	53.6%	-	Evolutionary Test-Time Compute
the ARCHitects (2024)	53.5%	-	Ganador Kaggle ARC Prize 2024
o3 (medium)	53%	\$2.52	-
SOAR	52%	-	Self-Improving Evolutionary Program Synthesis
TTT MIT/Cornell	47.5%	-	Test-Time Training
TRM (7M params)	45%	-	NEW Tiny Recursive Model - Paper Award 1st

Modelo/Sistema	Score	Costo/Tarea	Notas
o3 (low)	41%	\$1.22	-
Claude Sonnet 4 (16k)	40%	\$0.366	-
o4-mini (medium)	42%	\$0.23	-
Claude Opus 4 (16k)	35.7%	\$1.25	-
o1 (high)	35%	\$1.31	-
Gemini 2.5 Flash	33.3%	\$0.037	Mejor balance costo/rendimiento
Gemini 2.5 Pro	33%	\$0.569	-
o1 (medium)	31%	\$0.79	-
DeepSeek R1 (5/28)	21.2%	\$0.046	Actualizado
o4-mini (low)	21%	\$0.05	-
o1 (low)	20.5%	\$0.43	-
CompressARC (76K)	20-34%	-	Sin pretraining, MDL-based
Grok 3 Mini (low)	16.5%	\$0.0099	-
DeepSeek R1	15.8%	\$0.06	Open source
DeepSeek R1-Zero	14%	\$0.11	Sin SFT, solo RL
Claude 3.5 Sonnet	14%	-	Base, sin técnicas adicionales
Grok 3	5.5%	\$0.0093	-
GPT-4o	5%	-	Base LLM
Llama 4 Maverick	4.4%	\$0.0078	-
GPT-4	~0%	-	Base LLM
GPT-3	0%	-	Base LLM
Humanos	~85%	~\$5	Referencia humana

ARC-AGI-2 (lanzado Marzo 2025)

Modelo/Sistema	Score	Costo/Tarea	Notas
Poetiq (Gemini 3 Pro Ref)	54%	\$31	 SOTA refinement solution
Claude Opus 4.5 (64k)	37.6%	\$2.20	 SOTA comercial verificado
Gemini 3 Pro	31%	\$0.81	Baseline
NVARC (Kaggle 1st)	24.03%	\$0.20	 SOTA Kaggle 2025
the ARCHitects	16.53%	-	2D-aware masked-diffusion
MindsAI	12.64%	-	Test-time training
Claude Opus 4 (16k)	8.6%	\$1.93	-
TRM (7M params)	8%	-	 Tiny Recursive Model
o3 (high)	6.5%	\$0.834	-
o4-mini (high)	6.1%	\$0.856	-
Claude Sonnet 4 (16k)	5.9%	\$0.486	-
o3-pro (high)	4.9%	\$7.55	-
CompressARC (76K)	4%	-	Sin pretraining
Gemini 2.5 Pro	3.8%	\$0.813	-
o3 (medium)	2.9%	-	-
o4-mini (medium)	2.3%	-	-
o3 (low)	1.9%	-	-
o4-mini (low)	1.6%	-	-
DeepSeek R1	1.3%	\$0.08	-

Modelo/Sistema	Score	Costo/Tarea	Notas
Grok 3 Mini (low)	0.4%	\$0.013	-
Grok 3	0%	\$0.14	-
Llama 4 Maverick	0%	\$0.012	-
Humanos	>95%	-	Sin entrenamiento previo

Insights Clave (Actualizado 2025)

- Los LLMs base fracasan** en ARC-AGI porque dependen de memorización, no razonamiento adaptativo
 - Los Reasoning Models (o1, R1, o3)** mejoran mediante Chain-of-Thought, pero tienen límites
 - El test-time compute scaling** permite mejoras logarítmicas pero con rendimientos decrecientes
 - Tiny Recursive Models (TRM)** logran 45% en ARC-AGI-1 con solo 7M parámetros (<0.01% de LLMs)
 - ARC-AGI-2 sigue sin resolver:** incluso los mejores modelos <10%, humanos >95%
 - Refinement Loops** son el tema central de 2025 - mejoran performance iterativamente
 - La eficiencia importa:** Gemini 2.5 Flash ofrece el mejor ratio costo/rendimiento para ARC-AGI-1
 - No hay ganador claro:** cada modelo tiene trade-offs entre accuracy, costo y velocidad
 - ARC-AGI-3** (2026) introducirá razonamiento interactivo: exploración, planificación, memoria

Bibliografía Relevante

Papers con resultados ARC-AGI

- "Less is More: Recursive Reasoning with Tiny Networks"** (Oct 2025)
 - Autor: Alexia Jolicoeur-Martineau
 - arXiv: 2510.04871
 - Resultado: TRM (7M params) → ARC-AGI-1: 45%, ARC-AGI-2: 8%
 - 🏆 Paper Award 1st Place - ARC Prize 2025
 - "Small Language Models for Efficient Agentic Tool Calling"** (Dec 2025)
 - Autores: Jhandi, Kazi, Subramanian, Sendas
 - arXiv: 2512.15943
 - Resultado: OPT-350M fine-tuned → ToolBench: 77.55% (vs ChatGPT-CoT: 26%)
 - "On the Measure of Intelligence"** (2019)
 - Autor: François Chollet
 - arXiv: 1911.01547
 - Introduce ARC-AGI benchmark
 - "The Surprising Effectiveness of Test-Time Training for Abstract Reasoning"** (2024)
 - MIT/Cornell
 - Resultado: ARC-AGI-1: 47.5%

Enlaces imprescindibles

https://lewish.io/posts/arc-agi-2025-research-review#ttt-ttft

https://lewish.io/posts/how-to-beat-arc-agi-2

Tabla comparativa (Convergencias 2025 en ARC-AGI)

Fuentes (PDFs en este repo):

- ARC Prize 2025 Technical Report: [Doc/Bibliography/2601.10904v1.pdf](#)
 - HRM (Hierarchical Reasoning Model): [Doc/Bibliography/2506.21734v3.pdf](#)
 - TRM (Tiny Recursive Model): [Doc/Bibliography/2510.04871v1.pdf](#)
 - Thesis proposal (SLMs + program synthesis para ARC-AGI): [Doc/Master_Thesis.pdf](#)

Leyenda: ✓ = central / explícito en el enfoque; □ = aparece como técnica relacionada o compatible.

Técnica / Metodología (qué aporta)	Convergencia (forma del loop)	ARC Prize 2025 report	HRM (2506.21734)	TRM (2510.04871)	Thesis proposal
Refinement loop por tarea (iterar hasta cumplir demos)	Programa o modelo se refina con feedback	✓	□	✓	✓

Técnica / Metodología (qué aporta)	Convergencia (forma del loop)	ARC Prize 2025 report	HRM (2506.21734)	TRM (2510.04871)	Thesis proposal
Test-time compute scaling (más pasos = más fiabilidad)	Más iteraciones/muestras por task	✓	✓	✓	✓
Test-time training (TTT) / fine-tuning por puzzle	Refinamiento en espacio de pesos	✓	✓ (entrena desde cero por task)	✓ (entrena por task)	✓
Zero-pretraining (entrenar desde inicialización aleatoria)	Todo el “aprendizaje” sucede en test	✓	✓	✓	✓
Razonamiento iterativo en latente (estado que se actualiza)	Refinar estado/solución paso a paso	☐	✓	✓	✓
Deep supervision / multi-step improvement	Optimizar para mejorar en varios pasos	☐	✓	✓	☐
Halting / early-stopping aprendido (ACT o similar)	Ajustar compute a la dificultad	☐	✓	✓ (simplificado)	☐
Ensemble / voto sobre variantes	Explorar múltiples propuestas y seleccionar	✓	✓ (voto top-2 en ARC)	☐	☐
Data augmentation por simetrías 2D (dihedral, color perm, etc.)	Invariancias del dominio para generalizar	✓	✓	✓	☐
“Synthesis > prediction” (resolver generando procedimiento)	Output como resultado de un “programa”	✓	✓	✓	✓
Búsqueda + verificación (explore/verify)	Candidatos → score/verifier → refinar	✓	☐	☐	✓
Verificadores / harness de aplicación (layer externo)	Refinamiento a nivel orquestación	✓	☐	☐	☐
Regularización/criterios de compresión (MDL/description length)	Refina maximizando parsimonia	✓	☐	☐	☐

Dónde “vive” el refinamiento (vista unificadora)

Espacio	Qué se refina	Ejemplos típicos
Pesos (weight space)	Un solver “compilado” en pesos para ese puzzle	TTT / zero-pretraining (HRM, TRM), NVARC-style
Estado latente / solución parcial	La solución candidata y/o el estado de razonamiento	HRM (dos módulos), TRM (y + z)
Programas explícitos	Código (Python/DSL) o pseudo-programas	Evolución / program synthesis + verificación
“Programa en lenguaje”	CoT como traza optimizable con feedback	Reasoning models + verifiers + retries

Nota práctica (por qué converge todo)

En ARC-AGI-2 el patrón común es: *la generalización “sale” de iterar con feedback*, no solo de un pase directo. La diferencia principal entre líneas de trabajo es **dónde** iteran (pesos vs. programas vs. estados) y **qué feedback** usan (consistencia con demos, verificador, scoring, MDL, etc.).

PLATAFORMAS COMPUTACIÓN EN LA NUBE

google cloud azure

CÓMO LO VAMOS A VENDER? QUÉ ES LO QUE QUEREMOS CONSEGUIR? CÓMO NOS VAMOS A COMPARAR AL RESPECTO DE OTROS MODELOS? QUÉ MÉTRICAS DE EVALUACIÓN? MATRIZ PARA PONDERAR? O MULTIPLICADOR